

Guia de Desenvolvimento: Sistema de Gerenciamento de Biblioteca com IA

Introdução

Este guia tem como objetivo fornecer um passo a passo prático para o desenvolvimento de um sistema de gerenciamento de biblioteca comunitária utilizando **Java 21**, **Spring Boot**, **Hibernate**, **JPA** e **PostgreSQL** com apoio de inteligência artificial.

Considerações Iniciais

- Este guia assume que você está dando seus primeiros passos no desenvolvimento de software e tem noções básicas de lógica de programação.
 - O foco é a construção de um MVP (Produto Mínimo Viável) de um sistema de gerenciamento de biblioteca comunitária.
 - Em vez de apenas gerar código com Inteligência Artificial, o seu objetivo principal é compreender as decisões de projeto e atuar como um arquiteto e revisor crítico.
-

Fundamentos: Os Porquês do Nosso Projeto

Antes de abrirmos a IDE ou digitarmos prompts para a IA, precisamos entender **o que** estamos construindo e **por que** escolhemos cada uma das peças do nosso sistema. As decisões técnicas não são tomadas por "moda", mas sim para resolver problemas reais de engenharia, organização de equipe e manutenção a longo prazo.

1. Por que usar Docker e Docker Compose?

Se você já tentou rodar o projeto de um colega e ouviu a frase *"na minha máquina funciona, não sei por que na sua não está rodando"*, você já sentiu a dor que o **Docker** resolve.

O Problema: A inconsistência dos ambientes

Para rodar este MVP, precisamos de um banco de dados PostgreSQL ativo, de uma versão específica do Java (Java 21) e de dependências de sistema. Se cada estudante do CodeLab tiver que instalar

manualmente o PostgreSQL, configurar usuários, senhas, portas e variáveis de ambiente no Windows, Linux ou macOS, perderemos dias inteiros apenas configurando o ambiente. Além disso, pequenas diferenças de versão podem gerar bugs difíceis de rastrear.

A Solução: Containers

O Docker nos permite "empacotar" a nossa aplicação e suas dependências (como o banco de dados) em uma unidade isolada chamada **Container**.

- Imagine o container como uma **caixa de navio padronizada**: ela carrega tudo o que a aplicação precisa para rodar (código, runtime do Java, bibliotecas do sistema), independente de onde o navio (seu sistema operacional) esteja navegando.
- Uma **Imagem** é a receita dessa caixa (o molde). O **Container** é a caixa rodando na prática.

E o Docker Compose?

O nosso projeto não tem apenas uma peça. Ele tem o backend (Java/Spring), o banco de dados (PostgreSQL) e o frontend (Angular). O **Docker Compose** funciona como um **orquestrador ou gerente de condomínio**. Através de um único arquivo (`docker-compose.yml`), nós definimos como esses containers devem ser criados, quais portas eles vão expor para o seu computador e como eles vão se comunicar.

Com apenas um comando (`docker compose up`), todo o ecossistema do MVP sobe e se conecta automaticamente.

2. Por que usar um Banco de Dados Relacional (PostgreSQL)?

O banco de dados é a memória de longo prazo da nossa aplicação. Para a biblioteca, escolhemos o **PostgreSQL**, que é um banco de dados **relacional (SGBDR)**.

O que é um Banco Relacional?

Bancos relacionais organizam as informações em tabelas que se assemelham a planilhas muito inteligentes, compostas por:

- **Tabelas (Entities)**: Representam os conceitos do sistema (ex: `usuarios`, `livros`, `emprestimos`).
- **Linhas (Rows/Tuplas)**: São os registros de dados reais (ex: o usuário "João Silva", o livro "Dom Casmurro").
- **Colunas (Columns/Atributos)**: São as características do registro (ex: `email`, `isbn`, `data_cadastro`).
- **Chaves Primárias (PK - Primary Key)**: O identificador único de cada linha (garante que não existam dois registros idênticos).

- **Chaves Estrangeiras (FK - Foreign Key):** O elo que conecta uma tabela à outra (cria a relação).

Por que isso é perfeito para uma Biblioteca?

Uma biblioteca comunitária é movida a **relações**: um **usuário** faz um **empréstimo** de um **livro**.

Se utilizássemos um banco não-relacional (NoSQL, como MongoDB), onde os dados são salvos de forma mais livre e isolada, correríamos o risco de um estudante deletar o cadastro de um livro do acervo enquanto ele ainda constasse como emprestado para alguém. Isso geraria dados órfãos e relatórios inconsistentes.

O banco relacional se destaca por garantir as propriedades **ACID**:

- **Atomicidade:** Ou a transação acontece por inteiro (gravar o empréstimo E diminuir o estoque do livro), ou nada é salvo. Não há meio-termo.
- **Consistência:** O banco segue regras rígidas. Você não pode associar um empréstimo a um `usuario_id` que não existe.
- **Isolamento:** Vários usuários podem alugar livros ao mesmo tempo sem que uma transação interfira na outra.
- **Durabilidade:** Uma vez salvo, o dado não se perde, mesmo que ocorra uma queda de energia no servidor.

O **PostgreSQL** foi escolhido por ser o banco relacional gratuito e de código aberto mais robusto do mercado, sendo o padrão utilizado na imensa maioria das empresas de tecnologia.

3. Por que Java 21 e Spring Boot?

O ecossistema Java é um dos pilares do desenvolvimento corporativo global. Usamos o **Java 21** junto com o framework **Spring Boot 3**, pois é uma combinação popular, robusta e com uma vasta quantidade de recursos e documentação disponível, o que facilita o aprendizado e o desenvolvimento de aplicações web.

Por que Java (e por que a versão 21)?

1. **Tipagem Estática:** No Java, toda variável deve ter um tipo declarado (ex: `String`, `int`, `Book`). Se você tentar passar um texto onde o sistema espera um número, o compilador te avisa na hora, impedindo que esse erro chegue até o usuário final.
2. **Orientação a Objetos (POO):** O Java nos força a modelar o software pensando em "objetos" da vida real (`Usuario`, `Livro`, `Emprestimo`). Para estudantes, esta é a melhor forma de fixar conceitos de abstração, encapsulamento, herança e polimorfismo.
3. **Java 21 (LTS):** É uma versão de suporte de longo prazo extremamente moderna. Ela introduziu recursos como os **Records** (que reduzem drasticamente a verbosidade ao criar classes de transporte de dados) e melhorias imensas de performance.

Por que o Spring Boot?

Se você tentasse criar uma aplicação web usando Java puro, precisaria configurar manualmente um servidor web (como o Apache Tomcat), gerenciar conexões de rede HTTP, lidar com a conversão de JSON para objetos Java e criar toda a lógica de segurança. Isso exigiria centenas de linhas de código de configuração (chamado *boilerplate*).

O **Spring Boot** é um framework opinativo que resolve isso:

- **Servidor Embutido:** Ele já vem com o Tomcat configurado internamente. Basta rodar o projeto e o servidor web inicia sozinho na porta 8080.
 - **Injeção de Dependências:** Ele cuida de criar e gerenciar o ciclo de vida dos nossos objetos (Beans). Você não precisa ficar dando `new` a todo momento.
 - **Spring Data JPA & Hibernate:** Em vez de escrevermos consultas SQL complexas (`SELECT * FROM livros...`), o Spring Data traduz nossos métodos Java em comandos SQL compatíveis com o PostgreSQL de forma transparente.
-

4. Por que a Arquitetura em Camadas (e como ela funciona)?

Escrever todo o código do sistema em um único arquivo gigante tornaria a aplicação impossível de manter. Por isso, dividimos o backend do MVP em **Camadas**.

Para entender essa divisão, pense em um **Restaurante**:

O Garçom (Controller)

É a porta de entrada. O cliente (Frontend/Angular) envia uma requisição HTTP para uma rota (ex: `POST /book`). O Controller (Garçom) é responsável por atender essa chamada, verificar se a requisição está correta (validações do DTO) e passar o pedido para a cozinha (Service). **O garçom não cozinha!** Ele apenas atende e entrega as respostas HTTP (como `200 OK` ou `201 Created`).

O Cozinheiro (Service)

É o cérebro da aplicação. O Service (Cozinheiro) conhece as regras de negócio da biblioteca (a "receita"). É aqui que validamos se o usuário já estourou o limite de empréstimos, se o livro realmente está disponível, ou se a data de devolução é posterior à data atual. Ele pega as informações brutas, aplica as regras e prepara o resultado.

O Despenseiro (Repository)

O Repository é quem tem a chave da despensa (o banco de dados). O Service não sabe mexer diretamente no PostgreSQL; ele apenas pede ao Repository: *"Busque o livro com o ID X para mim"* ou

"Salve este novo registro". Graças ao Spring Data JPA, essa camada se comunica diretamente com o Hibernate para buscar e salvar os dados nas tabelas.

O Prato Feito (DTO - Data Transfer Object)

No restaurante, o cliente não quer ver a cozinha bagunçada ou ter acesso a ingredientes crus que estão no estoque. Ele quer o prato pronto e embalado.

O **DTO** é essa embalagem personalizada. Ele garante que:

1. Não vazemos informações confidenciais do banco de dados (como senhas criptografadas ou IDs internos desnecessários).
 2. O Frontend receba exatamente o formato de dados que ele precisa para renderizar a tela, de forma limpa e otimizada.
 3. Possamos ter um DTO específico para entrada (ex: `CreateBookDTO`, contendo apenas título e autor) e outro para saída (ex: `BookDTO`, contendo ID, título, autor e categorias).
-

5. Por que Angular + TailwindCSS no Frontend? (Visão Geral)

Embora o foco deste guia seja o backend, o MVP conta com um frontend em **Angular** e **TailwindCSS** para que os usuários finais interajam com o sistema.

Por que Angular?

O Angular é um framework frontend corporativo mantido pelo Google. Diferente de bibliotecas menores, ele é uma solução completa (*batteries included*). Ele traz uma estrutura rígida baseada em **Componentes** e **TypeScript** (que traz para o navegador a mesma segurança de tipagem que o Java nos dá no backend).

Em uma fábrica de software como o CodeLab, onde estudantes entram e saem de projetos ao final de cada semestre, a vigidez estrutural do Angular garante que qualquer pessoa consiga entender onde estão as telas, as rotas e os serviços frontend do projeto sem dificuldades.

Por que TailwindCSS?

Em vez de escrevermos arquivos de estilização CSS gigantes (`index.css` lotado de classes customizadas difíceis de nomear e organizar), o TailwindCSS nos fornece **classes utilitárias prontas** direto no HTML (ex: `class="flex items-center justify-between p-4 bg-blue-600 rounded-lg text-white"`). Isso agiliza o design do MVP, permitindo que a interface fique moderna, responsiva e bonita rapidamente.

Como o Frontend e o Backend se conversam?

Eles se comunicam por meio de requisições **HTTP** assíncronas utilizando o padrão **REST**. O backend expõe uma documentação viva (gerada pelo **OpenAPI/Scalar**). O frontend lê esse "contrato" e faz chamadas enviando e recebendo dados em formato **JSON** (JavaScript Object Notation), que é a linguagem universal de troca de dados na web moderna.

Dicas Gerais para o Uso de IA

Forneça Contexto

IAs não adivinham a arquitetura. Diga sempre qual é a linguagem, o framework e a versão (ex: Java 21, Spring Boot).

Seja Específico nas Restrições

Se você não quer usar uma biblioteca específica, avise no prompt.

Desconfie do "Código Pronto"

A IA tenta resolver o problema da forma mais rápida, não necessariamente da forma mais arquiteturalmente correta para a manutenção de longo prazo. **O desenvolvedor é o arquiteto; a IA é apenas uma assistente de digitação super rápida.**

1 - Sugestão de Modelagem de Dados

IA Utilizada: Gemini 3.1 PRO

Prompt

Estou desenvolvendo um sistema para gerenciamento de uma biblioteca. A primeira versão do sistema deverá garantir as operações essenciais para o funcionamento da biblioteca:

- Cadastro e gestão de livros no acervo: Permitirá a inserção de novos títulos, edição de informações básicas (título, autor, categoria) e a exclusão ou baixa de livros danificados/perdidos.

- Registro e controle de empréstimos: Possibilitará vincular um livro específico a um usuário cadastrado, definindo a data de retirada e a data limite para devolução.
- Listagem em tempo real dos livros disponíveis: Oferecerá uma tela de busca e visualização rápida dos títulos que estão fisicamente na biblioteca e prontos para novos empréstimos, separando-os dos que já estão emprestados.
- Cadastro de usuários (com foco em núcleos familiares): Permitirá o registro ágil das famílias da comunidade, armazenando dados básicos de contato para facilitar a comunicação e o acompanhamento dos livros retirados.

Considerando que vou usar Java 21 + spring boot + Hibernate e JPA com postgres, quero que me de a sugestão da modelagem do banco de dados para a fase de MVP.

Resposta da IA

Para um MVP de um sistema de biblioteca comunitária utilizando Java 21, Spring Boot e PostgreSQL, a modelagem deve focar na simplicidade para garantir agilidade, mas manter a integridade referencial para permitir o crescimento futuro (roadmap).

Modelagem do Banco de Dados (Entidades)

Tabela: **livros** (Books)

Armazena o acervo da biblioteca.

- **id** : UUID (PK) – Uso de UUID é recomendado para evitar previsibilidade e facilitar integrações
- **titulo** : VARCHAR(255)(Not Null)
- **autor** : VARCHAR(255)
- **isbn** : VARCHAR(13)(Unique)
- **categoria** : VARCHAR(100)
- **status** : ENUM(DISPONIVEL, EMPRESTADO, DANIFICADO, EXTRAVIADO)
- **data_cadastro** : TIMESTAMP

Tabela: **usuarios** (Users)

Foco em núcleos familiares, conforme o escopo.

- `id` : UUID (PK)
- `nome_completo` : VARCHAR(255)(Not Null)
- `email` : VARCHAR(150)(Unique, Nullable — nem todos na comunidade podem ter e-mail)
- `telefone` : VARCHAR(20)(Not Null — essencial para cobrança)
- `endereco` : TEXT
- `data_cadastro` : TIMESTAMP

Tabela: `emprestimos` (Loans)

A tabela que faz a intersecção do sistema.

- `id` : UUID (PK)
- `livro_id` : UUID (FK → `livros.id`)
- `usuario_id` : UUID (FK → `usuarios.id`)
- `data_emprestimo` : DATE (Not Null)
- `data_devolucao_prevista` : DATE (Not Null)
- `data_devolucao_efetiva` : DATE (Nullable)
- `status` : ENUM(ATIVO, DEVOLVIDO, ATRASADO)

Análise Crítica

É importante destacar que é **sempre necessário julgar a resposta da IA**.

Nesse caso, por exemplo, optamos por:

- **Não utilizar UUID:** Um número sequencial (inteiro) seria mais simples e eficiente para o contexto de um sistema de biblioteca comunitária, onde a escalabilidade não é uma preocupação imediata.
- **Campo status dinâmico:** O campo `status` da tabela de empréstimos poderia ser calculado dinamicamente com base nas datas de empréstimo e devolução, eliminando a necessidade de armazenar esse campo no banco de dados e o trabalho desnecessário de sempre manter esse estado atualizado.
- **Tabela de categorias separada:** Optamos por criar uma tabela a parte para categorias, pois por serem entidades semanticamente diferentes, é melhor que estejam desacopladas nas regras de negócio.

2 – Dockerização do Projeto

Prompt 1: Docker Compose

Gere um yml de docker-compose para meu backend em Java 21 com Postgres

Resposta: docker-compose.yml

```
services:
  db:
    image: postgres:16-alpine
    container_name: library_postgres
    restart: always
    environment:
      POSTGRES_DB: dlibrary
      POSTGRES_USER: dlibrary
      POSTGRES_PASSWORD: dlibrary
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

  backend:
    build:
      context: ./backend # Aponta para a pasta onde está o Dockerfile do
    container_name: library_backend
    ports:
      - "8080:8080" # Expõe a porta 8080 para a sua máquina
    depends_on:
      - db # Garante que o back-end só inicie depois do banco de dados
    environment:
      # Sobrescreve as configurações do seu application.yml
      - SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/dlibrary
      - SPRING_DATASOURCE_USERNAME=dlibrary
      - SPRING_DATASOURCE_PASSWORD=dlibrary

volumes:
  postgres_data:
```

Observação: Apenas tivemos que nos atentar às credenciais do banco de dados e mudar a porta a ser exposta.

Prompt 2: Dockerfile

Gere um Dockerfile para o meu backend em Java 21 com Spring Boot

Resposta: Dockerfile

```
# Estágio 1: Build (Compilação com Maven)
FROM maven:3.9.6-eclipse-temurin-21-alpine AS builder
WORKDIR /app

# Copia apenas o pom.xml primeiro para aproveitar o cache do Docker
COPY pom.xml .

# Baixa as dependências (isso acelera builds futuros)
RUN mvn dependency:go-offline

# Copia o código-fonte
COPY src ./src

# Compila o projeto ignorando os testes (para ser mais rápido no ambiente)
RUN mvn clean package -DskipTests

# Estágio 2: Execução (Apenas o JRE, sem o Maven)
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app

# Pega o .jar gerado no Estágio 1 e renomeia para app.jar
COPY --from=builder /app/target/*.jar app.jar

# Expõe a porta que o Spring Boot usa
EXPOSE 8080

# Comando para iniciar a aplicação
ENTRYPOINT ["java", "-jar", "app.jar"]
```

3 - Implementando Autenticação com JWT

O que é JWT (JSON Web Token)?

De forma resumida, o **JWT** é um padrão para autenticação **stateless** (sem estado). Em vez de o servidor guardar na memória quem está logado (usando sessões), ele gera um token criptografado contendo as informações do usuário e uma data de validade.

O cliente (frontend) guarda esse token e o envia no cabeçalho (Header) de toda requisição. O servidor apenas valida a assinatura do token e libera o acesso.

Para o nosso MVP da biblioteca, isso é excelente, pois facilita a integração futura com aplicações mobile ou web independentes.

Prompt

IA Utilizada: Gemini 3.1 PRO

Estou desenvolvendo o backend de um sistema de biblioteca comunitária em Java 21 com Spring Boot 3.

Preciso implementar um sistema de login e registro de usuários usando JWT.

Minha entidade de usuário já está mapeada e tenho um UserRepository.

Não quero criar filtros complexos na mão (OncePerRequestFilter), quero usar as abordagens mais modernas

do Spring Security 6 com oauth2ResourceServer.

Como configuro o SecurityConfig, a geração do token e os endpoints de autenticação de forma simples para um MVP?

Resumo da Resposta

A IA geralmente fornecerá uma estrutura dividida em algumas classes chave. No nosso projeto, o resultado esperado e implementado foi dividido nas seguintes partes:

- **SecurityConfig.java:** Configura as rotas públicas (como `/auth/login`, `/auth/register`, `/book`) e as privadas. Configura também os Beans de criptografia de senha (`BCryptPasswordEncoder`) e o gerenciamento de autenticação.
- **TokenService.java:** Responsável exclusivamente por montar e assinar o JWT definindo o emissor, o tempo de expiração e as permissões (roles).

- **AuthController.java**: Expõe os endpoints REST para receber as credenciais e devolver o token gerado.
- **JpaUserDetailsService.java**: Ensina o Spring Security a buscar os usuários no nosso banco de dados (PostgreSQL) usando o nosso UserRepository.

Onde Aplicar o Senso Crítico

Ao pedir para uma IA gerar configurações de segurança, ela tomará decisões arquiteturais por você. É crucial analisar o que foi entregue:

Geração das Chaves de Assinatura (RSA vs HMAC)

No nosso código, optamos por usar chaves **assimétricas (RSA)** geradas dinamicamente na inicialização da classe SecurityConfig (`KeyPairGenerator.getInstance("RSA")`).

O lado bom: Para um MVP ou ambiente local, é perfeito. Você não precisa se preocupar em criar, guardar e carregar arquivos de chave privada/pública no seu computador.

O alerta crítico: Se você levar esse exato código para produção, toda vez que o servidor reiniciar, um novo par de chaves será gerado. Isso significa que todos os tokens JWT já emitidos e na mão dos usuários se tornarão inválidos instantaneamente, forçando todos a fazerem login novamente.

Em um ambiente real, a IA precisaria ser instruída a ler essas chaves de um cofre de senhas ou de variáveis de ambiente fixas.

Separação de Responsabilidades

Muitas vezes a IA tenta colocar a lógica de geração de token e de registro de usuários diretamente dentro do AuthController.

No nosso projeto, **intervimos e separamos as responsabilidades** criando um `TokenService` e um `RegistrationService`. Isso mantém os Controllers focados apenas em receber requisições e devolver respostas HTTP, facilitando a manutenção.

Validade do Token

No `TokenService`, o tempo de expiração foi definido no código (`expiresAt(now.plus(1, ChronoUnit.HOURS))`).

Em sistemas reais, a IA não sabe qual o tempo ideal para o seu negócio. Um token de biblioteca que expira em 1 hora faz sentido? Talvez o usuário precise se logar muitas vezes.

O desenvolvedor deve ajustar esse valor ou extrair-lo para o arquivo `application.yml` para facilitar alterações sem precisar recompilar o código.

4 – Implementando uma Nova Funcionalidade: Cadastro de Livros

Quando adicionamos uma nova funcionalidade no sistema, como o cadastro de livros, precisamos que a informação atravesse várias camadas da nossa arquitetura até chegar ao banco de dados.

Para entender como pedir ajuda à IA nesse processo, primeiro precisamos entender a responsabilidade de cada peça que compõe essa funcionalidade.

A Responsabilidade de Cada Camada

Controller (BookController)

É a porta de entrada. Ele não toma decisões complexas. Apenas recebe a requisição HTTP (ex: um POST para `/book`), verifica se o usuário tem permissão (usando anotações como `@PreAuthorize("hasRole('ADMIN')")`), repassa os dados para o Service e devolve a resposta HTTP adequada (ex: 200 OK ou 201 Created).

Service (BookService)

É o cérebro da aplicação. Aqui ficam as regras de negócio. No nosso exemplo, é o Service que busca as categorias no banco, calcula a lógica entre o total de cópias (`copies`) e as cópias disponíveis para empréstimo (`availableCopies`), e prepara o objeto para ser salvo.

Repository (BookRepository)

É o comunicador com o banco de dados. Graças ao Spring Data JPA, na maioria das vezes, é apenas uma interface vazia que já herda os métodos de salvar, buscar e deletar.

Mapper (BookMapper)

Responsável por traduzir as Entidades do banco de dados para os DTOs (Data Transfer Objects) que serão enviados ao cliente, garantindo que não vamos vaziar informações sensíveis do banco na resposta

da API.

O Padrão de DTOs Utilizado

Para o tráfego de dados, adotamos uma estratégia de segregação através de **Records do Java**, criando um DTO específico para cada intenção:

CreateBookDTO

Usado exclusivamente para receber os dados de criação. Ele contém as anotações de validação rigorosas, como `@NotBlank` para título e `@ISBN` para garantir a integridade do código do livro. Não possui o campo `id`, pois o livro ainda não existe.

UpdateBookDTO

Usado para atualizações. Pode ter regras de validação diferentes do `Create` (por exemplo, permitir campos nulos caso o usuário não queira atualizar todas as informações de uma vez).

BookDTO

Usado como retorno da API. Este contém o `id` gerado pelo banco e formata os dados para exibição (trazendo a lista de `CategoryDTO` ao invés de apenas os IDs).

Benefícios dessa Abordagem

- **Desacoplamento e Segurança:** Se a regra para criar um livro mudar, você altera apenas o `CreateBookDTO` sem quebrar a visualização ou a atualização
- **Clareza:** Fica extremamente claro para quem lê o código qual é o formato de entrada e de saída de cada endpoint

Abordagem Alternativa

Outro padrão comum é criar uma única classe "Wrapper" de DTOs usando **nested records**. Por exemplo, uma classe `BookRequest` contendo `public record Create(...)` e `public record Update(...)` dentro dela. Isso diminui a quantidade de arquivos no projeto, mantendo o agrupamento semântico, mas a lógica de separação de intenções continua a mesma.

Prompt

IA Utilizada: Gemini 3.1 PRO

Tenho uma entidade 'Book' e 'Category' no meu projeto Spring Boot com Java 21.

Quero criar a funcionalidade de cadastro e listagem de livros.

Já tenho a interface BookRepository. Siga a arquitetura de Controller, Service e Mapper.

Crie os métodos para salvar um novo livro e listar os livros com paginação.

Utilize o padrão de records do Java para criar um 'CreateBookDTO' com validações do Jakarta Validation (título obrigatório, ISBN válido) e um 'BookDTO' de retorno.

Apenas usuários com a role ADMIN podem criar livros.

Onde Aplicar o Senso Crítico

Ao receber o código da IA, não basta copiar e colar. Veja o que precisa ser analisado e ajustado:

Regras de Negócio Ocultas

A IA vai gerar um `BookService.create()` simples, apenas copiando os dados do DTO para a Entidade e salvando. Porém, no nosso sistema, o número de cópias iniciais é igual ao número de cópias disponíveis.

Intervenção: Fomos nós que tivemos que intervir e escrever

`book.setAvailableCopies(dto.copies())`; no Service. A IA não adivinha regras de negócio específicas da sua biblioteca.

Relacionamentos

Ao receber os `categoriesIds` no DTO, a IA pode tentar salvar o livro diretamente ou sugerir iterações ineficientes no banco de dados.

Intervenção: No nosso código, o senso crítico ditou injetar o `CategoryService` para buscar todas as categorias de uma vez de forma segura

(`categoryService.findAllByIdIn(dto.categoriesIds())`) antes de vincular ao livro.

5 - Entendendo a Paginação e a Recepção de Dados no Controller

Quando construímos uma API REST, o Controller é a porta de entrada. Para que ele saiba o que o cliente (frontend) está pedindo, ele precisa extrair informações da requisição HTTP.

No Spring Boot, fazemos isso mapeando os argumentos dos métodos.

Três Formas de Receber Dados

@RequestBody (Corpo da Requisição)

Usado para receber cargas de dados maiores, geralmente em formato JSON. É utilizado em requisições POST (criar) e PUT (atualizar).

Exemplo no código: `create(@RequestBody CreateBookDTO dto)` . O Spring pega o JSON enviado e transforma no nosso record `CreateBookDTO` .

@PathVariable (Variável de Caminho)

Usado para extrair um valor que faz parte da URL (da rota) do endpoint. Serve para identificar um recurso específico.

Exemplo no código: `update(..., @PathVariable Long id)` . Se a requisição for para `PUT /book/5` , o Spring entende que o `id` é `5` .

Parâmetros de Consulta (Query Params) e o Pageable

Valores enviados após o ponto de interrogação na URL (ex: `/book?page=0&size=10`). O Spring poderia receber isso usando `@RequestParam` , mas ele oferece uma interface poderosa chamada `Pageable` .

Exemplo no código: `getAll(Pageable pageable)` . Quando o cliente acessa `/book?page=0&size=10&sort=title,asc` , o Spring automaticamente agrupa essas informações (página 0, 10 itens por página, ordenado por título) dentro do objeto `pageable` e repassa para o Service.

Por Que Usar Paginação?

Imagine se a biblioteca tiver 5.000 livros cadastrados. Um simples `GET /book` tentaria buscar e devolver todos os 5.000 de uma vez. Isso:

- Consome muita memória do servidor
- Sobrecarrega o banco de dados
- Deixa o aplicativo do usuário lento

A paginação resolve isso trazendo os dados em **"pacotes" (páginas)**. É justamente isso que acontece em sites de comércio eletrônico, redes sociais e, claro, em bibliotecas online. O cliente pode navegar por páginas de resultados sem travar o sistema.

Prompt

IA Utilizada: Gemini 3.1 PRO

No meu controller de livros em Spring Boot, preciso criar um endpoint GET para listar todos os livros.

No entanto, não quero retornar uma List simples, quero usar paginação para não sobrecarregar o banco de dados.

Como configuro o método no BookController, no BookService e no BookRepository para receber os parâmetros de página e tamanho e retornar os dados paginados?

Meu retorno deve ser um DTO e não a Entidade.

Julgando a Resposta

A paginação do Spring Data JPA é muito robusta, mas a IA pode sugerir implementações que deixam a desejar em alguns detalhes de arquitetura:

Vazamento de Entidade na Paginação

É extremamente comum a IA gerar o Service devolvendo `Page<Book>` (a Entidade do banco) direto para o Controller. Isso quebra o nosso padrão de DTOs. O Spring oferece um método `.map()` excelente dentro da própria interface `Page`.

Intervenção: No BookService, fizemos exatamente isso:

`bookRepository.findAll(pageable).map(BookMapper::toDto);` . Assim, pegamos a página de Entidades que veio do banco e convertemos cada item interno para `BookDTO` antes de devolver ao Controller.

A IA muitas vezes tenta fazer iterações manuais complexas (for ou `stream().map()`) que não são necessárias aqui.

O Tipo de Retorno (Page vs List)

Preste atenção no tipo de retorno que a IA sugere. Se ela sugerir retornar `List<BookDTO>`, você perde as vantagens da paginação no frontend.

Ao retornar `Page<BookDTO>`, o Spring Boot constrói um JSON muito rico. Ele não manda apenas a lista de livros, mas também **metadados essenciais**, como:

- `totalElements` : Total geral de livros no banco
- `totalPages` : Quantas páginas existem
- `number` : A página atual

Isso é fundamental para que quem for construir a interface do usuário consiga desenhar aqueles botões de "Próxima Página" e "Página Anterior".

Tratamento de Ordenação Padrão

A IA pode apenas injetar o `Pageable`. Caso o cliente faça uma requisição para `/book` sem informar os parâmetros `?page=0&size=10`, o Spring assume um padrão (geralmente página 0, tamanho 20).

Como desenvolvedor, você pode querer forçar um limite ou uma ordenação caso o cliente não envie nada (ex: usar a anotação `@PageableDefault(size = 10, sort = "title")` no Controller).

A IA raramente sugere isso proativamente.

6 – Spring: Ciclo de Vida, Beans e Injeção de Dependência

Onde Essas Classes São Instanciadas?

Como o Controller sabe onde encontrar o Service se não foi feito nenhum `new Service()` ?

Para desmistificar isso, vamos usar o nosso domínio de **Categorias** (Category), que possui uma estrutura limpa e direta, para entender como o Spring gerencia a vida útil dos componentes da aplicação.

O Ponto de Entrada e o "Radar" do Spring

Toda aplicação Spring Boot tem uma classe principal anotada com `@SpringBootApplication`. Quando você roda a aplicação, essa anotação liga um **"radar"** (chamado Component Scan). Esse radar vasculha todas as pastas do seu projeto procurando por classes que tenham **"etiquetas"** (estereótipos) dizendo que elas devem ser gerenciadas pelo framework.

Os Estereótipos (Anotações de Camada)

No nosso código de Categoria, usamos três anotações principais para etiquetar nossas classes:

- `@RestController` na classe `CategoryController`
- `@Service` na classe `CategoryService`
- A interface `CategoryRepository` estende `JpaRepository`, o que o Spring já entende automaticamente como um repositório de dados

O Que São Beans e o Padrão Singleton

Quando o radar do Spring encontra essas anotações, ele cria uma única instância de cada uma dessas classes na memória durante a inicialização do sistema. Esses objetos gerenciados pelo Spring são chamados de **Beans**.

Isso significa que o padrão adotado é o **Singleton**. Se 1.000 usuários tentarem buscar categorias ao mesmo tempo, todos eles vão passar pela mesma instância de `CategoryController` e pela mesma instância de `CategoryService`.

É por isso que você nunca deve guardar "estado" (como uma variável que guarda o usuário atual) dentro de um Service ou Controller, pois essa informação vazaria entre requisições diferentes.

Injeção de Dependência

Como o Spring já criou e guardou essas instâncias na memória, nós não precisamos (e não devemos) instanciá-las manualmente. Em vez de fazer `CategoryRepository repo = new CategoryRepository()` dentro do Service, nós pedimos para o Spring **"injetar"** essa dependência.

Veja como isso foi feito no `CategoryService`:

```
@Service
public class CategoryService {
```

```
private final CategoryRepository categoryRepository;

// O Spring injeta o Repository existente automaticamente ao chamar e
public CategoryService(CategoryRepository categoryRepository) {
    this.categoryRepository = categoryRepository;
}
// ...
}
```

Prompt

IA Utilizada: Gemini 3.1 PRO

Estou criando a camada de Service e Controller para a entidade 'Category' no Spring Boot.

Já possuo a interface CategoryRepository.

Gere o CategoryService e o CategoryController básicos com injeção de dependência via construtor, seguindo as melhores práticas.

Julgando a Resposta da IA

A Injeção de dependência é um dos pontos onde a IA mais sugere práticas ultrapassadas se você não for específico.

A "Armadilha" do @Autowired em Atributos (Field Injection)

Se você pedir um código sem especificar, a esmagadora maioria das IAs vai gerar algo assim:

```
@Service
public class CategoryService {
    @Autowired
    private CategoryRepository categoryRepository;
}
```

Intervenção e Crítica

O uso de `@Autowired` direto no atributo (**Field Injection**) não é mais recomendado pela própria equipe do Spring. Ele:

- Dificulta a criação de testes unitários (pois você precisaria de reflexão ou do contexto do Spring para injetar um mock)
- Permite que o objeto seja criado com estado inválido (sem suas dependências)

O Jeito Certo (Injeção por Construtor)

O código que implementamos no projeto (e que devemos cobrar da IA nos prompts) usa **Injeção por Construtor**. Ao declarar o atributo como `private final` e criar um construtor recebendo a dependência, nós garantimos que:

- O `CategoryService` nunca existirá sem um `CategoryRepository` válido
- A variável é `final`, então ninguém pode sobrescrevê-la acidentalmente durante a execução do programa
- Fica extremamente fácil instanciar a classe manualmente nos testes unitários passando um repositório "falso" (Mock) no construtor

7 - Mapeando Relacionamentos e Regras Transversais (JPA/Hibernate)

Em bancos de dados relacionais, as tabelas não vivem isoladas.

O domínio de **Borrowing** (Empréstimo) é o exemplo perfeito de uma **entidade de intersecção**: ela existe exclusivamente para conectar um Usuário (User) a um Livro (Book) em um determinado momento.

Quando usamos o JPA/Hibernate, não lidamos diretamente com chaves estrangeiras (IDs) no código Java, mas sim com os objetos completos. O framework faz a "**tradução**" disso para o banco de dados através de anotações específicas.

Entendendo as Anotações de Relacionamento

No nosso código da entidade Borrowing, utilizamos as seguintes abordagens:

@ManyToOne (Muitos para Um)

Indica a cardinalidade da relação. **"Muitos"** empréstimos podem estar associados a **"Um"** único usuário, e **"Muitos"** empréstimos podem envolver **"Um"** mesmo livro ao longo do tempo.

@JoinColumn(name = "user_id")

Diz exatamente qual será o nome da coluna física na tabela do banco de dados (PostgreSQL) que guardará a chave estrangeira (FK).

@UniqueConstraint

Uma excelente prática de integridade aplicada no topo da classe (`@Table`). Ela garante que o banco de dados bloqueie qualquer tentativa de registrar o mesmo usuário pegando o mesmo livro exatamente no mesmo instante (`borrowed_at`), prevenindo duplicações acidentais por cliques duplos no frontend.

Prompt

IA Utilizada: Gemini 3.1 PRO

Estou criando a entidade 'Borrowing' (Empréstimo) no meu projeto Spring Boot usando JPA e Hibernate.

Ela deve se relacionar com as entidades 'User' e 'Book' que já existem.

Gere a classe Borrowing. O empréstimo deve ter data de retirada, data limite de devolução, data de devolução efetiva e a condição do livro devolvido.

Crie também o repository e o service básicos para realizar um empréstimo e uma devolução.

Julgando a Resposta da IA

A entidade de intersecção é onde as IAs mais costumam gerar **"armadilhas arquiteturais"** e falhar na lógica de negócio. Fique atento a estes **três pontos críticos** ao avaliar o código gerado:

O Perigo do CascadeType.ALL

A IA quase sempre adiciona algo como `@ManyToOne(cascade = CascadeType.ALL)` . Isso é **extremamente perigoso**.

O "Cascade" dita que as operações feitas no empréstimo devem se propagar para as entidades relacionadas. Se a IA colocar `CascadeType.ALL` ou `CascadeType.REMOVE`, significa que, se você deletar um registro de empréstimo, o Hibernate apagará o Usuário e o Livro do banco de dados junto com ele!

No nosso código, omitimos o cascade de propósito, mantendo os registros independentes.

Regras de Negócio Transversais (O Efeito Colateral)

Ao pedir o `BorrowingService`, a IA provavelmente fará um método `borrow()` que apenas instancia o Borrowing, seta o usuário, o livro e salva no banco.

Intervenção: Empréstimo de um livro afeta o estoque! No nosso código, o `BorrowingService` não age sozinho. Ele chama `bookService.borrowBook(book)` para garantir que a lógica de subtrair as cópias disponíveis seja respeitada. A IA raramente conecta domínios diferentes sem ser forçada a isso.

Lógica Condicional Rica

Na devolução (`returnBorrow`), a IA faria apenas `borrowing.setReturnedAt(now)`.

Intervenção: Adicionamos o senso crítico humano. O nosso sistema avalia o estado do livro devolvido: `if (!"DANIFICADO".equalsIgnoreCase(dto.bookCondition()))`. Se o livro voltou danificado, a cópia não volta para as cópias disponíveis da biblioteca.

A IA cria códigos "felizes"; o desenvolvedor cria códigos para o mundo real.

8 – Documentação da API: O Contrato entre Cliente e Servidor

Desenvolver os endpoints e a regra de negócio é apenas metade do trabalho. A outra metade é garantir que quem for construir a interface do usuário (o frontend) saiba exatamente como se comunicar com o seu sistema.

Em arquiteturas modernas, o backend e o frontend vivem em mundos separados. **A documentação é a ponte oficial entre eles.**

Se o formato de um DTO mudar ou uma rota exigir um novo parâmetro, e isso não estiver documentado de forma clara, o frontend inevitavelmente vai quebrar. É por isso que adotamos o padrão **OpenAPI**.

O Que é a Especificação OpenAPI

A **OpenAPI** (antigamente conhecida como Swagger) é uma especificação padrão da indústria para descrever APIs RESTful. Ela define um formato universal (geralmente em JSON ou YAML) que lista:

- Todas as rotas disponíveis
- Quais métodos HTTP elas aceitam
- Quais os formatos de entrada (DTOs de request)
- Quais os formatos de saída (DTOs de response)
- Os requisitos de segurança

A Geração do JSON

No Spring Boot, não precisamos escrever esse arquivo JSON na mão. Ao adicionar uma biblioteca como o **springdoc-openapi**, o Spring vasculha o nosso código em tempo de execução. Ele lê as nossas anotações (`@RestController` , `@GetMapping` , `@RequestBody`), olha para os nossos Records de DTOs e gera esse descritivo OpenAPI de forma totalmente automatizada.

A Interface Swagger UI / Scalar UI

Ler um arquivo JSON gigante não é humano. É aí que entram ferramentas como o **Swagger UI** ou o **Scalar**. Elas pegam aquele JSON gerado pelo Spring e o transformam em uma página web interativa e visualmente agradável.

Lá, o desenvolvedor pode ler as descrições e até mesmo testar as requisições com um botão **"Try it out"**, direto do navegador.

Configuração no Projeto

No nosso sistema, deixamos as rotas do Swagger e do Scalar públicas lá no `SecurityConfig` para facilitar o acesso dos desenvolvedores. Além disso, na classe principal `LibraryApplication` , ativamos o suporte web do Spring Data para serializar as páginas via DTO (`PageSerializationMode.VIA_DTO`).

Isso é crucial para que o gerador da OpenAPI entenda corretamente a estrutura do nosso retorno paginado e exiba o **"contrato" certo** na interface gráfica.

Prompt

IA Utilizada: Gemini 3.1 PRO

Estou usando Spring Boot 3 e Java 21 e quero documentar minha API REST usando o springdoc-openapi.

Minha API é protegida por JWT (Bearer token). Gere uma classe de configuração 'OpenApiConfig' para definir o título 'API - Sistema de Biblioteca Comunitária', a versão '1.0 (MVP)' e a descrição.

É vital que você configure o SecurityScheme para que o Swagger UI exiba o botão de 'Authorize' e permita enviar o token JWT nas requisições de teste.

Julgando a Resposta da IA

Documentar com IA geralmente envolve gerar a classe de configuração global e, opcionalmente, adicionar anotações nos Controllers. Fique atento a estes pontos:

O Botão de Autorização (SecurityScheme)

O erro mais comum ao pedir essa configuração para a IA é ela gerar apenas as informações básicas (Título e Versão), esquecendo que a API é trancada por senha.

Intervenção: No nosso `OpenApiConfig`, garantimos a inclusão da anotação `@SecurityScheme(type = SecuritySchemeType.HTTP, scheme = "bearer", bearerFormat = "JWT")` e adicionamos o `.addSecurityItem(...)` globalmente.

Se a IA não trouxer isso, o Swagger UI será gerado com sucesso, mas você receberá erro 403 Forbidden ao tentar testar qualquer rota privada por não ter onde colar o seu token.

Poluição Visual nos Controllers

A IA pode sugerir que você encha todos os seus Controllers com anotações verbosas como `@Operation`, `@ApiResponse`, etc. Para um MVP, isso muitas vezes polui a legibilidade do código.

O Springdoc já faz um trabalho excelente inferindo 90% da documentação apenas lendo o código puro. O senso crítico aqui é **saber dosar**: deixe o Spring fazer o trabalho sujo automaticamente e use anotações manuais apenas em endpoints muito complexos onde o retorno não seja óbvio.

Vazamento de Informações Sensíveis

Verifique se a interface do Swagger não está expondo acidentalmente DTOs internos ou Entidades do banco de dados. Como adotamos rigorosamente o padrão de `CreateBookDTO`, `UpdateBookDTO` e `BookDTO`, a nossa documentação gerada exibirá exatamente (e apenas) o que o cliente pode enviar e receber, provando o valor da separação de camadas.

Conclusão

Conclusão: A IA como Ferramenta e o Desenvolvedor como Arquiteto

Este guia apresentou o ciclo completo de desenvolvimento de uma API RESTful, abrangendo desde a concepção e modelagem inicial do banco de dados até a definição do contrato de comunicação por meio da especificação OpenAPI. Fica evidente que o ecossistema Java, estruturado sobre o Spring Boot e o Java 21, fornece um conjunto de ferramentas robustas e consolidadas para a construção de sistemas escaláveis e arquiteturalmente maduros.

A adoção da Inteligência Artificial no processo de desenvolvimento de software impõe uma mudança significativa de paradigma. A IA atua como um recurso altamente eficiente para a mitigação do trabalho repetitivo, sendo capaz de gerar boilerplate, estruturar configurações e propor esqueletos de código de forma célere. Contudo, é imperativo reconhecer que essas ferramentas carecem de contexto sistêmico e de domínio do negócio. Elas não possuem a capacidade inerente de discernir impactos operacionais complexos — como a diferença entre reintegrar ou descartar um livro danificado no acervo — tampouco avaliam adequadamente as implicações de segurança ao gerenciar chaves criptográficas em ambientes produtivos.

Consequentemente, a aplicação das diretrizes aqui expostas eleva o papel do programador da simples escrita de código para a posição de Arquiteto de Software e Revisor Crítico. O artefato gerado pela máquina deve ser tratado estritamente como um ponto de partida. A intervenção humana permanece indispensável para aplicar a correta injeção de dependências, isolar o tráfego de dados por meio de DTOs e orquestrar de maneira segura as regras de negócio transversais, assegurando assim a integridade e a manutenibilidade do sistema.

Por fim, espera-se que este documento consolide uma base técnica sólida para a construção do MVP do Sistema de Biblioteca Comunitária, servindo como um material de alinhamento arquitetural. A Inteligência Artificial deve, portanto, ser explorada como um vetor de produtividade, exigindo sempre que o domínio absoluto sobre a arquitetura e as decisões técnicas do projeto permaneça sob a responsabilidade dos desenvolvedores.

Lembre-se: A IA é uma ferramenta poderosa, mas o **senso crítico do desenvolvedor** é essencial para tomar as melhores decisões arquiteturais para seu projeto.